

VERILOG EXPERIMENTS

Experiment No. 1

Familiarization of Verilog

Digital systems are highly complex. At their most detailed level, they may consist of millions of elements, such as a collection of logic gates or transistors. Hardware description languages have evolved to aid in the design of systems with this large number of elements and wide range of electronic and logical abstractions. Hardware description languages have been developed for modeling and simulating hardware functions.

Difference between standard programming languages and hardware description languages:

- Standard programming languages: sequential
- HDLs: describe parallel and concurrent behavior

There are two standard HDLs that are supported by IEEE: VHDL and Verilog HDL. The HDL used in our lab will be Verilog.

INTRODUCTION TO VERILOG HDL

Verilog is a Hardware Description Language (HDL) which is used to model electronic systems. It provides the designer entry into the world of large, complex digital systems design. The Verilog language provides the digital system designer with a means of describing a digital system at a wide range of levels of abstraction, and, at the same time, provides access to computer-aided design tools to aid in the design process at these levels. It also fulfils the need for verifying the design for functionality and timing constraints like propagation delay setup and hold times. The components of the target design can be described at different levels with the help of constructs in Verilog.

1. Circuit Level/Switch Level: MOS switch is the basic element which can be used to build basic circuits like inverters, logic gates, 1-bit dynamic and static memories.
2. Gate Level or structural level: Design is carried out in terms of basic gates. All basic gates are available as ready modules called "primitives". Primitives can be incorporated into design descriptions directly.
3. Data Flow Level: All possible operations on signals and variables are represented in terms of assignments.
4. Behavioral Level: This level describes a system by concurrent algorithms and the design description looks like a C program. Compactness and the comprehensive nature of the design description make the development process fast and efficient. Functions, Tasks and always blocks are the main elements.

LANGUAGE CONSTRUCTS AND CONVENTIONS IN VERILOG

Verilog has its own constructs and conventions. Any source file in Verilog is made up of number of ASCII characters. These characters are grouped into sets - referred to as "lexical tokens". Verilog has 7 types of lexical tokens - operators, keywords, identifiers, white spaces, comments, numbers and strings. Verilog is case sensitive, so all keywords are in lower case.

White Space Characters

Blanks (\b), tabs (\t), newlines (\n) and form feed form the white space characters in Verilog. In any design description the white space characters are included to improve readability. White space characters have significance only when they appear in string.

Comments

Comments are incorporated in two ways.

// begins a single line comment, terminated by a newline.

/* begins a multi-line block comment, terminated by a */.

For Example

Module D_FF (Q, DP, CLK); //This is the design of a D flip-flop.

/* Here Q is the output. DP is the input and CLK is the clock. */

Keywords

The keywords define the language constructs. All keywords in Verilog are in small letters and require to be used as such.

Some of the examples are:

module → signifies the beginning of a module definition.

endmodule → signifies the end of module definition.

begin → signifies the beginning of block of statements.

end → signifies the end of a block of statements.

if → signifies a conditional activity to be checked.

assign → assigns a value or an expression to a net or variable.

Identifiers

Identifiers are user-defined words for variables, function names, module names, block names and instance names. Identifiers begin with a letter or underscore and can include any number of letters, digits and underscores. Identifiers in Verilog are case sensitive.

Numbers

The numbers can be of integer type or real type.

Integer Numbers

Number storage is defined as a number of bits, but values can be specified in binary, octal, decimal or hexadecimal. The representation has three tokens with an optional sign preceding it. Numbers may be sized or unsized. Unsized integers default to at least 32 bits.

Syntax: size'base value

Integer	Stored As	Description
1	00000000000000000000000000000001	Unsize 32 bits
8'h AA	10101010	Sized hexadecimal
3'b001	001	3 bit number
9'o123	001 010 011	9 bit octal number
9'h?A	zzzzz1010	5 bit hex number

String

A string is a sequence of characters enclosed by double quotes. It must be contained on a single line. Verilog treats a string as a sequence of ASCII characters.

Logical Values

Verilog uses a 4 value logic system for modelling.

Logical Value	Description
0	Zero , low or false
1	One, high or true
Z or z	High impedance (tri-state or floating)
X or x	Unknown or uninitialized (don't care)

Data Types

Verilog has two major data type classes:

- 1) Net Data type
- 2) Variable data type

Nets: Net data types are used to make connections between parts of a design. Nets reflect the value and strength level of the drivers of the net or the capacitance of the net, and do not have a value of their own. A net can be specified in different ways.

Wire

A wire (or net) represents a physical wire in a circuit and is used to connect gates or modules. The value of a wire can be read, but not assigned to, in a function or block. A wire does not store its value but must be driven by a continuous assignment statement or by connecting it to the output of a gate or module.

Syntax: `wire [msb: lsb] wire_variable_list;`

Example

```
wire c; //declare a wire c
```

Variable data types

Variable data types are used as temporary storage of programming data. Variables can only be assigned a value from within an initial procedure, an always procedure, a task or a function. Variables can only store logic values; they cannot store logic strength. Variables are un-initialized at the start of simulation, and will contain logic X until a value is assigned. Variables can be declared through a keyword `reg`.

Reg

A reg (register) is a data object that holds its value from one procedural assignment to the next. They are used only in functions and procedural blocks. A reg is a Verilog variable type and does not necessarily imply a physical register.

Syntax: `reg [msb: lsb] reg_variable_list;`

Example

```
reg a; // single 1-bit register variable
```

```
reg [7:0] r_vector; // an 8-bit vector, a bank of 8 registers.
```

Scalars and Vectors

Entities representing single bits are called “scalars”. Often multiple lines carry signals in a cluster like data bus or address bus. The group of regs is treated as “vector”.

Verilog Module

Verilog language uses a hierarchical, functional unit-based design approach. The whole design consists of several smaller modules. The complexity of the modules is decided by the designer.

Verilog module:

- Definition of the input and output ports
- Definition of the logical relationship between the input and output ports

Port declaration syntax:

<direction> <data_type> <size> <port_name>;

Direction:

- Input port: input
- Output port: output
- Bi-directional: inout

Verilog language uses a hierarchical, functional unit based design approach:

- The whole design consists of several smaller modules
- The complexity of the modules is decided by the designer

Module Syntax:

```
module module_name (port_list);  
    input [msb:lsb] input_port_list;  
    output [msb:lsb] output_port_list;  
    inout [msb:lsb] inout_port_list;  
    ... statements ...  
Endmodule
```

Operators

Arithmetic Operators

These perform arithmetic operations.

+ (addition)
- (subtraction)
* (multiplication)
/ (division)
%(modulus)

Relational Operators

Relational operators compare two operands and return a single bit 1 or 0. These operators synthesize into comparators.

< (less than)
<= (less than or equal to)
> (greater than)
>= (greater than or equal to)

== (equal to)

!= (not equal to)

Bit-wise Operators

Bit-wise operators do a bit-by-bit comparison between two operands

~ (bitwise NOT) & (bitwise AND)

| (bitwise OR)

^ (bitwise XOR)

~^ or ^~ (bitwise XNOR)

Logical Operators

Logical operators return a single bit 1 or 0. They are the same as bit-wise operators only for single bit operands. They can work on expressions, integers or groups of bits, and treat all values that are nonzero as "1". Logical operators are typically used in conditional (if ... else) statements since they work with expressions.

! (logical NOT)

&& (logical AND)

|| (logical OR)

Concatenation Operator

The concatenation operator combines two or more operands to form a larger vector.

{ } (concatenation)

Experiment No. 2

Basic Gates in Verilog

Aim

To write Verilog descriptions for AND, OR, NOT, NAND, NOR gates.

Programs

AND Gate

Gate Level Modelling

```
module andgate1(c, a, b);  
input a, b;  
output c;  
and x1(c, a, b);  
endmodule
```

Data Flow Modelling

```
module andgate1(c, a, b);  
input a, b;  
output c;  
assign c = (a & b);  
endmodule
```

Behavioral Modelling

```
module andgate1(c, a, b);  
input a, b;  
output reg c;  
always @ (a, b)  
c = (a & b);  
endmodule
```

Testbench

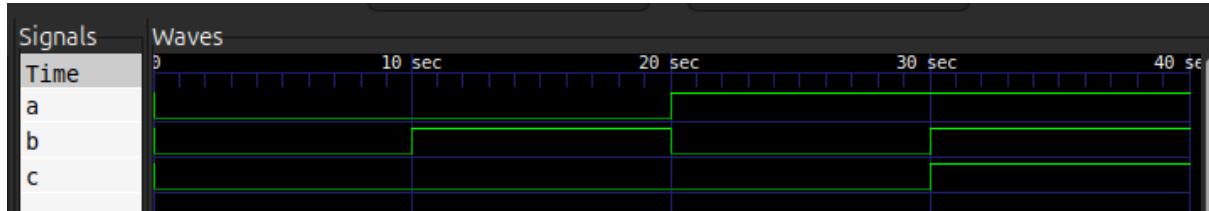
```
module testbench1;  
reg a, b;  
wire c;  
andgate1 uut(c, a, b);  
initial begin  
$dumpfile("andgate1.vcd");  
$dumpvars();  
a=0; b=0; #10  
a=0; b=1; #10  
a=1; b=0; #10  
a=1; b=1; #10
```

```

$finish;
end
endmodule

```

Output



OR Gate

Gate Level Modelling

```

module orgate1(c, a, b);
input a, b;
output c;
or x1(c, a, b);
endmodule

```

Data Flow Modelling

```

module orgate1(c, a, b);
input a, b;
output c;
assign c = (a | b);
endmodule

```

Behavioral Modelling

```

module orgate1(c, a, b);
input a, b;
output reg c;
always @ (a, b)
c = (a | b);
endmodule

```

Testbench

```

module testbench1;
reg a, b;
wire c;
orgate1 uut(c, a, b);
initial begin
$dumpfile("orgate1.vcd");
$dumpvars();
a=0; b=0; #10
a=0; b=1; #10

```

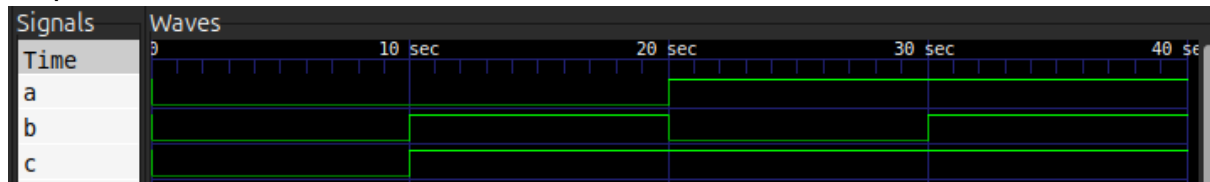


```

a=1; b=0; #10
a=1; b=1; #10
$finish;
end
endmodule

```

Output



NOT Gate

Gate Level Modelling

```

module notgate1(c, a);
input a;
output c;
not x1(c, a);
endmodule

```

Data Flow Modelling

```

module notgate1(c, a);
input a;
output c;
assign c = ~a;
endmodule

```

Behavioral Modelling

```

module notgate1(c, a);
input a;
output reg c;
always @ (a)
c = ~a;
endmodule

```

Testbench

```

module testbench1;
reg a;
wire c;
notgate1 uut(c, a);
initial begin
$dumpfile("notgate1.vcd");
$dumpvars();
a=0; #10

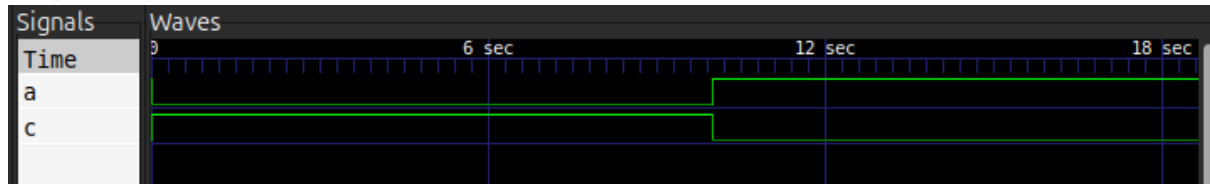
```

```

a=1; #10
$finish;
end
endmodule

```

Output



NAND Gate

Gate Level Modelling

```

module nandgate1(c, a, b);
input a, b;
output c;
nand x1(c, a, b);
endmodule

```

Data Flow Modelling

```

module nandgate1(c, a, b);
input a, b;
output c;
assign c = (a ~& b);
endmodule

```

Behavioral Modelling

```

module nandgate1(c, a, b);
input a, b;
output reg c;
always @ (a, b)
c = (a ~& b);
endmodule

```

Testbench

```

module testbench1;
reg a, b;
wire c;
nandgate1 uut(c, a, b);
initial begin
$dumpfile("nandgate1.vcd");
$dumpvars();
a=0; b=0; #10
a=0; b=1; #10

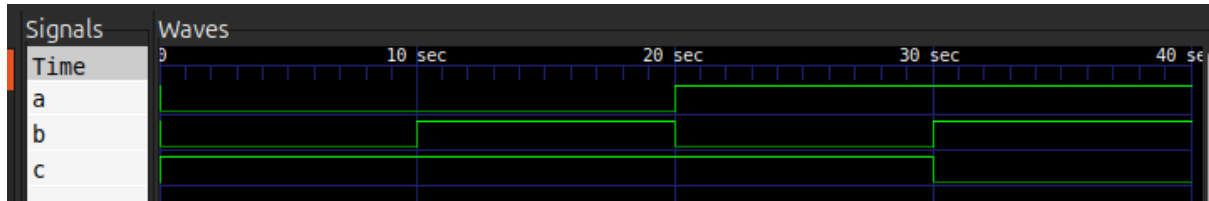
```

```

a=1; b=0; #10
a=1; b=1; #10
$finish;
end
endmodule

```

Output



NOR Gate

Gate Level Modelling

```

module norgate1(c, a, b);
input a, b;
output c;
nor x1(c, a, b);
endmodule

```

Data Flow Modelling

```

module norgate1(c, a, b);
input a, b;
output c;
assign c = (a ~| b);
endmodule

```

Behavioral Modelling

```

module norgate1(c, a, b);
input a, b;
output reg c;
always @ (a, b)
c = (a ~| b);
endmodule

```

Testbench

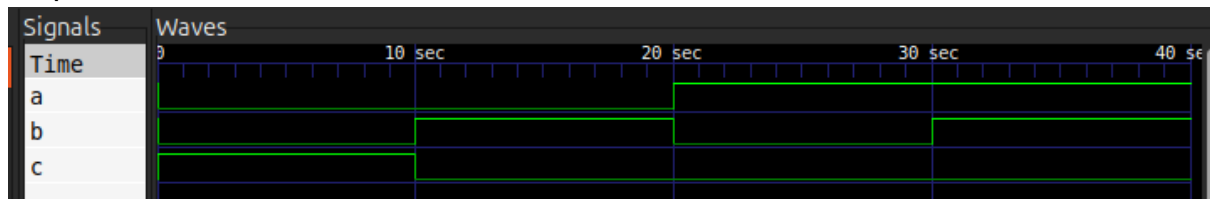
```

module testbench1;
reg a, b;
wire c;
norgate1 uut(c, a, b);
initial begin
$dumpfile("norgate1.vcd");
$dumpvars();

```

```
a=0; b=0; #10
a=0; b=1; #10
a=1; b=0; #10
a=1; b=1; #10
$finish;
end
endmodule
```

Output



Result

The Verilog descriptions for AND, OR, NOT, NAND, NOR gates have been successfully written and outputs verified.

Experiment No. 3

Adders using Verilog

Aim

To write Verilog descriptions for half adder and full adder.

Programs

Half Adder

Gate Level Modelling

```
module halfadder(sum, carry, a, b);
input a, b;
output sum, carry;
xor x1(sum, a, b);
and a1(carry, a, b);
endmodule
```

Data Flow Modelling

```
module halfadder(sum, carry, a, b);
input a, b;
output sum, carry;
assign sum = (a ^ b);
assign carry = (a & b);
endmodule
```

Behavioral Modelling

```
module halfadder(sum, carry, a, b);
input a, b;
output reg sum, carry;
always @ (a, b)
sum = (a ^ b);
always @ (a, b)
carry = (a & b);
endmodule
```

Testbench

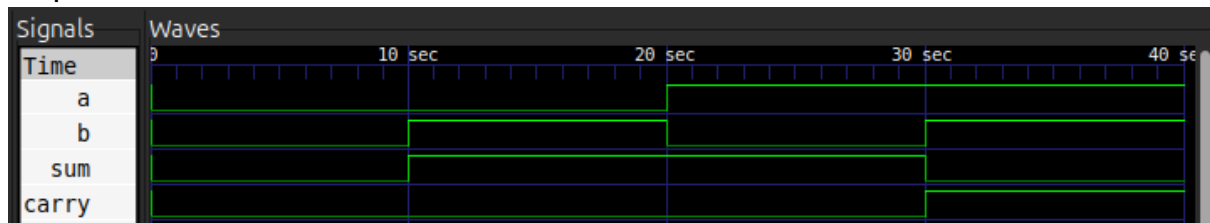
```
module testbench1;
reg a, b;
wire sum, carry;
halfadder uut(sum, carry, a, b);
initial begin
$dumpfile("halfadder.vcd");
$dumpvars();
a=0; b=0; #10
```

```

a=0; b=1; #10
a=1; b=0; #10
a=1; b=1; #10
$finish;
end
endmodule

```

Output



Full Adder

Gate Level Modelling

```

module fulladder(Sum, Carry, A, B, Cin);
input A, B, Cin;
wire w1, w2, w3;
output Sum, Carry;
xor x1(w1, A, B);
xor x2(Sum, w1, Cin);
and a1(w2, A, B);
and a2(w3, w1, Cin);
or o1(Carry, w2, w3);
endmodule

```

Data Flow Modelling

```

module fulladder(Sum, Carry, A, B, Cin);
input A, B, Cin;
output Sum, Carry;
assign Sum = A ^ B ^ Cin;
assign Carry = ((A&B)|(A&Cin)|(B&Cin));
endmodule

```

Behavioral Modelling

```

module fulladder(Sum, Carry, A, B, Cin);
input A, B, Cin;
output reg Sum, Carry;
always@(A, B, Cin)
begin
Sum = (A ^ B ^ Cin);
Carry = ((A&B)|(A&Cin)|(B&Cin));
end

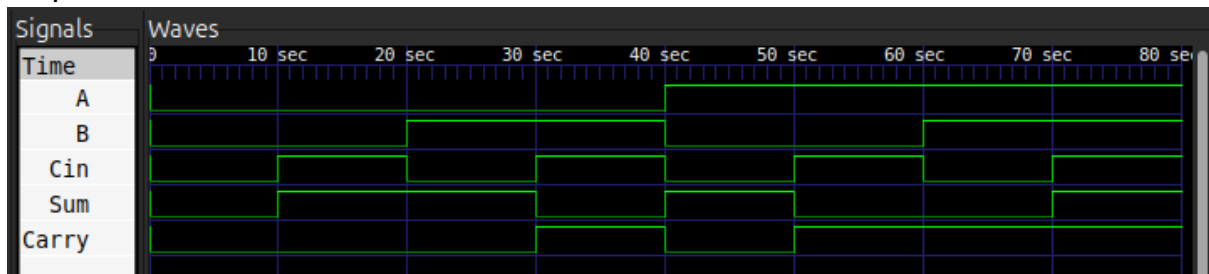
```

```
endmodule
```

Testbench

```
module testbench1;  
  reg A, B, Cin;  
  wire Sum, Carry;  
  fulladder uut(Sum, Carry, A, B, Cin);  
  initial begin  
    $dumpfile("fulladder.vcd");  
    $dumpvars();  
    A = 0; B = 0; Cin = 0; #10  
    A = 0; B = 0; Cin = 1; #10  
    A = 0; B = 1; Cin = 0; #10  
    A = 0; B = 1; Cin = 1; #10  
    A = 1; B = 0; Cin = 0; #10  
    A = 1; B = 0; Cin = 1; #10  
    A = 1; B = 1; Cin = 0; #10  
    A = 1; B = 1; Cin = 1; #10  
    $finish;  
  end  
endmodule
```

Output



Result

The Verilog descriptions for half adder and full adder have been successfully written and outputs verified.

Experiment No. 4

Subtractors using Verilog

Aim

To write Verilog descriptions for half subtractor and full subtractor.

Programs

Half Subtractor

Gate Level Modelling

```
module halfsubtractor(difference, borrow, a, b);  
input a, b;  
wire w1;  
output difference, borrow;  
xor x1(difference, a, b);  
not n1(w1, a);  
and a1(borrow, w1, b);  
endmodule
```

Data Flow Modelling

```
module halfsubtractor(difference, borrow, a, b);  
input a, b;  
output difference, borrow;  
assign difference = a ^ b;  
assign borrow = ((~a) & b);  
endmodule
```

Behavioral Modelling

```
module halfsubtractor(difference, borrow, a, b);  
input a, b;  
output reg difference, borrow;  
always @ (a, b)  
begin  
difference = a ^ b;  
borrow = ((~a) & b);  
end  
endmodule
```

Testbench

```
module testbench1;  
reg a, b;  
wire difference, borrow;  
halfsubtractor uut(difference, borrow, a, b);  
initial begin  
$dumpfile("halfsubtractor.vcd");
```

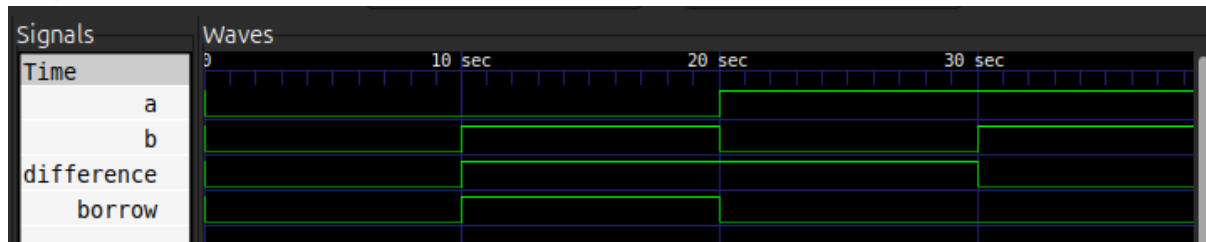


```

$dumpvars();
a = 0; b = 0; #10
a = 0; b = 1; #10
a = 1; b = 0; #10
a = 1; b = 1; #10
$finish;
end
endmodule

```

Output



Full Subtractor

Gate Level Modelling

```

module fullsubtractor(Difference, Borrow, A, B, Bin);
input A, B, Bin;
wire w1, w2, w3, w4, w5;
output Difference, Borrow;
xor x1(w1, A, B);
xor x2(Difference, w1, Bin);
not n1(w2, A);
and a1(w3, w2, B);
not n2(w4, w1);
and a2(w5, w4, Bin);
or o1(Borrow, w3, w5);
endmodule

```

Data Flow Modelling

```

module fullsubtractor(Difference, Borrow, A, B, Bin);
input A, B, Bin;
output Difference, Borrow;
assign Difference = A ^ B ^ Bin;
assign Borrow = ((Bin & ~(A ^ B))) | ((~A) & B);
endmodule

```

Behavioral Modelling

```

module fullsubtractor(Difference, Borrow, A, B, Bin);
input A, B, Bin;
output reg Difference, Borrow;
always@(A, B, Bin)

```

```

begin
Difference = (A ^ B ^ Bin);
Borrow = ((Bin&~(A ^ B))|((~A)&B));
end
endmodule

```

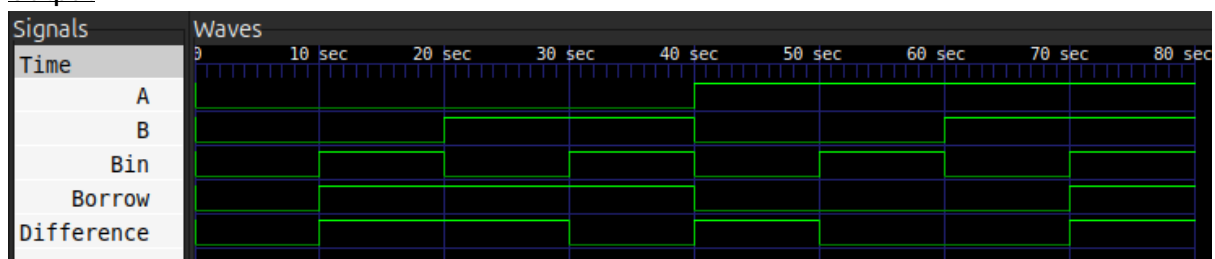
Testbench

```

module testbench1;
reg A, B, Bin;
wire Difference, Borrow;
fullsubtractor uut(Difference, Borrow, A, B, Bin);
initial begin
$dumpfile("fullsubtractor.vcd");
$dumpvars();
A = 0; B = 0; Bin = 0; #10
A = 0; B = 0; Bin = 1; #10
A = 0; B = 1; Bin = 0; #10
A = 0; B = 1; Bin = 1; #10
A = 1; B = 0; Bin = 0; #10
A = 1; B = 0; Bin = 1; #10
A = 1; B = 1; Bin = 0; #10
A = 1; B = 1; Bin = 1; #10
$finish;
end
endmodule

```

Output



Result

The Verilog descriptions for half subtractor and full subtractor have been successfully written and outputs verified.

Experiment No. 5

Multiplexer using Verilog

Aim

To write Verilog description for 4:1 multiplexer (MUX).

Programs

Gate Level Modelling

```
module mux(y, i0, i1, i2, i3, s0, s1);
input i0, i1, i2, i3, s0, s1;
output y;
wire n0, n1, a0, a1, a2, a3;
not x1(n0, s0);
not x2(n1, s1);
and x3(a0, n1, n0, i0);
and x4(a1, n1, s0, i1);
and x5(a2, s1, n0, i2);
and x6(a3, s1, s0, i3);
or x7(y, a0, a1, a2, a3);
endmodule
```

Data Flow Modelling

```
module mux(y, i0, i1, i2, i3, s0, s1);
input i0, i1, i2, i3, s0, s1;
output y;
assign y = (((~s0)&(~s1)&i0)|(s0&(~s1)&i1)|((~s0)&s1&i2)|(s0&s1&i3));
endmodule
```

Behavioral Modelling

```
module mux(y,s0,s1,i0,i1,i2,i3);
input s0,s1,i0,i1,i2,i3;
output reg y;
always @ (s0,s1,i0,i1,i2,i3)
y = (((~s0)&(~s1)&i0)|(s0&(~s1)&i1)|((~s0)&s1&i2)|(s0&s1&i3));
endmodule
```

Testbench

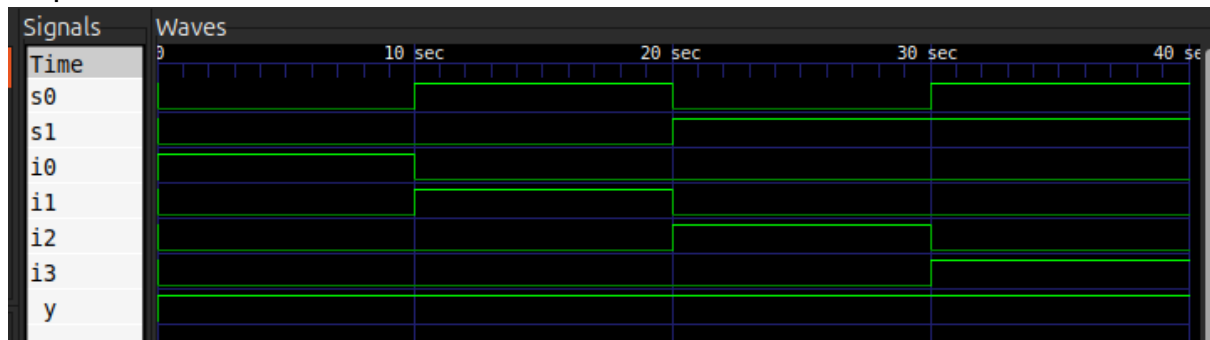
```
module testbench;
reg s0, s1, i0, i1, i2, i3;
wire y;
mux uut(y,i0,i1,i2,i3,s0,s1);
initial begin
$dumpfile("mux.vcd");
$dumpvars();
s1 = 0; s0 = 0; i0 = 1; i1 = 0; i2 = 0; i3 = 0; #10
```

```

s1 = 0; s0 = 1; i0 = 0; i1 = 1; i2 = 0; i3 = 0; #10
s1 = 1; s0 = 0; i0 = 0; i1 = 0; i2 = 1; i3 = 0; #10
s1 = 1; s0 = 1; i0 = 0; i1 = 0; i2 = 0; i3 = 1; #10
$finish;
end
endmodule

```

Output



Result

The Verilog descriptions for 4:1 multiplexer (MUX) has been successfully written and outputs verified.

Experiment No. 6

Code Converters using Verilog

Aim

To write Verilog description for 4-bit binary to gray and gray to binary code converters.

Programs

Binary to Gray

Gate Level Modelling

```
module btog (b1, b2, b3, b0, g1, g2, g3, g0);
input b1, b2, b3, b0;
output g1, g2, g3, g0;
xor (g0, b0, b1);
xor (g1, b1, b2);
xor (g2, b2, b3);
buf(g3, b3);
endmodule
```

Data Flow Modelling

```
module btog (b1, b2, b3, b0, g1, g2, g3, g0);
input b1, b2, b3, b0;
output g1, g2, g3, g0;
xor (g0, b0, b1);
xor (g1, b1, b2);
xor (g2, b2, b3);
assign g3 = b3;
endmodule
```

Behavioral Modelling

```
module btog (b1, b2, b3, b0, g1, g2, g3, g0);
input b1, b2, b3, b0;
output reg g1, g2, g3, g0;
always @(*)
begin
g0 = b0 ^ b1;
g1 = b1 ^ b2;
g2 = b2 ^ b3;
g3 = b3;
end
endmodule
```

Testbench

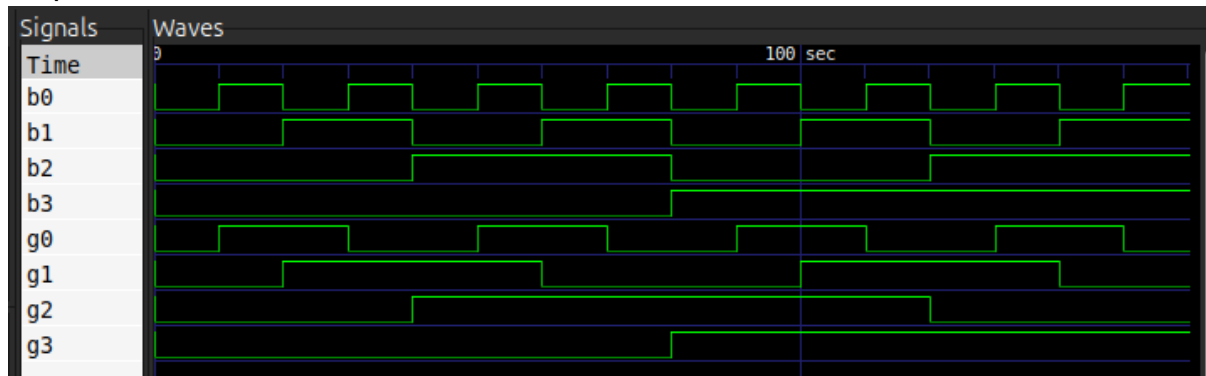
```
module testbtog;
reg b1, b2, b3, b0;
```

```

wire g1, g2, g3, g0;
btog uut(.b0(b0), .b1(b1), .b2(b2), .b3(b3), .g0(g0), .g1(g1), .g2(g2), .g3(g3));
initial begin
$dumpfile("btog.vcd");
$dumpvars();
b3=0; b2=0; b1=0; b0=0; #10;
b3=0; b2=0; b1=0; b0=1; #10;
b3=0; b2=0; b1=1; b0=0; #10;
b3=0; b2=0; b1=1; b0=1; #10;
b3=0; b2=1; b1=0; b0=0; #10;
b3=0; b2=1; b1=0; b0=1; #10;
b3=0; b2=1; b1=1; b0=0; #10;
b3=0; b2=1; b1=1; b0=1; #10;
b3=1; b2=0; b1=0; b0=0; #10;
b3=1; b2=0; b1=0; b0=1; #10;
b3=1; b2=0; b1=1; b0=0; #10;
b3=1; b2=0; b1=1; b0=1; #10;
b3=1; b2=1; b1=0; b0=0; #10;
b3=1; b2=1; b1=0; b0=1; #10;
b3=1; b2=1; b1=1; b0=0; #10;
b3=1; b2=1; b1=1; b0=1; #10;
$finish;
end
endmodule

```

Output



Gray to Binary

Gate Level Modelling

```

module gtob(g3, g2, g1, g0, b3, b2, b1, b0);
input g3, g2, g1, g0;
output b3, b2, b1, b0;
buf(b3, g3);
xor(b2, g2, b3);
xor(b1, g1, b2);
xor(b0, g0, b1);

```

endmodule

Data Flow Modelling

```
module gtob(g3, g2, g1, g0, b0, b1, b2, b3);
input g3, g2, g1, g0;
output b0, b1, b2, b3;
assign b3 = g3;
xor(b2, g2, b3);
xor(b1, b2, g1);
xor(b0, b1, g0);
endmodule
```

Behavioral Modelling

```
module gtob(g3, g2, g1, g0, b3, b2, b1, b0);
input g3, g2, g1, g0;
output reg b3, b2, b1, b0;
always @(*)
begin
b3 = g3;
b2 = b3 ^ g2;
b1 = b2 ^ g1;
b0 = b1 ^ g0;
end
endmodule
```

Testbench

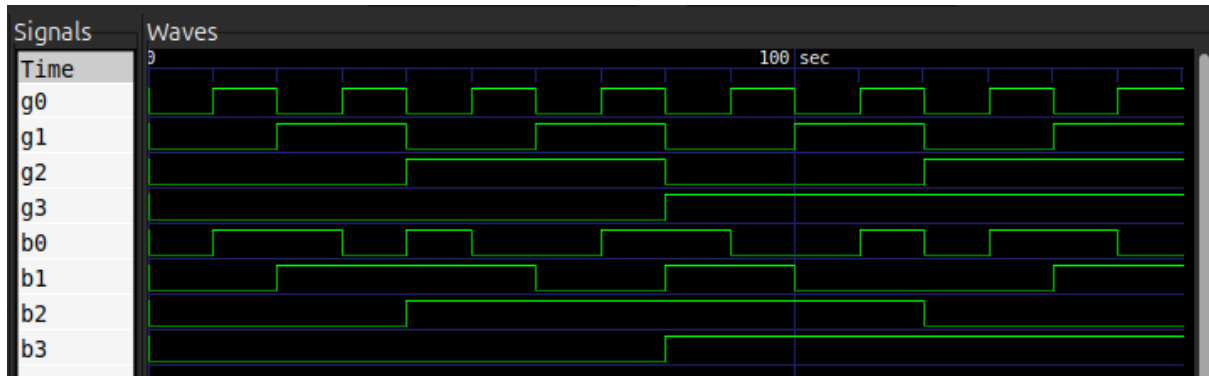
```
module testgtob;
reg g0, g1, g2, g3;
wire b0, b1, b2, b3;
gtob uut(.g0(g0), .g1(g1), .g2(g2), .g3(g3), .b0(b0), .b1(b1), .b2(b2), .b3(b3));
initial begin
$dumpfile("gtob.vcd");
$dumpvars();
g3 = 0; g2 = 0; g1 = 0; g0 = 0; #10;
g3 = 0; g2 = 0; g1 = 0; g0 = 1; #10;
g3 = 0; g2 = 0; g1 = 1; g0 = 0; #10;
g3 = 0; g2 = 0; g1 = 1; g0 = 1; #10;
g3 = 0; g2 = 1; g1 = 0; g0 = 0; #10;
g3 = 0; g2 = 1; g1 = 0; g0 = 1; #10;
g3 = 0; g2 = 1; g1 = 1; g0 = 0; #10;
g3 = 0; g2 = 1; g1 = 1; g0 = 1; #10;
g3 = 1; g2 = 0; g1 = 0; g0 = 0; #10;
g3 = 1; g2 = 0; g1 = 0; g0 = 1; #10;
g3 = 1; g2 = 0; g1 = 1; g0 = 0; #10;
g3 = 1; g2 = 0; g1 = 1; g0 = 1; #10;
end
```

```

g3 = 1; g2 = 1; g1 = 0; g0 = 0; #10;
g3 = 1; g2 = 1; g1 = 0; g0 = 1; #10;
g3 = 1; g2 = 1; g1 = 1; g0 = 0; #10;
g3 = 1; g2 = 1; g1 = 1; g0 = 1; #10;
$finish;
end
endmodule

```

Output



Result

The Verilog descriptions for 4-bit binary to gray and gray to binary code converters have been successfully written and outputs verified.